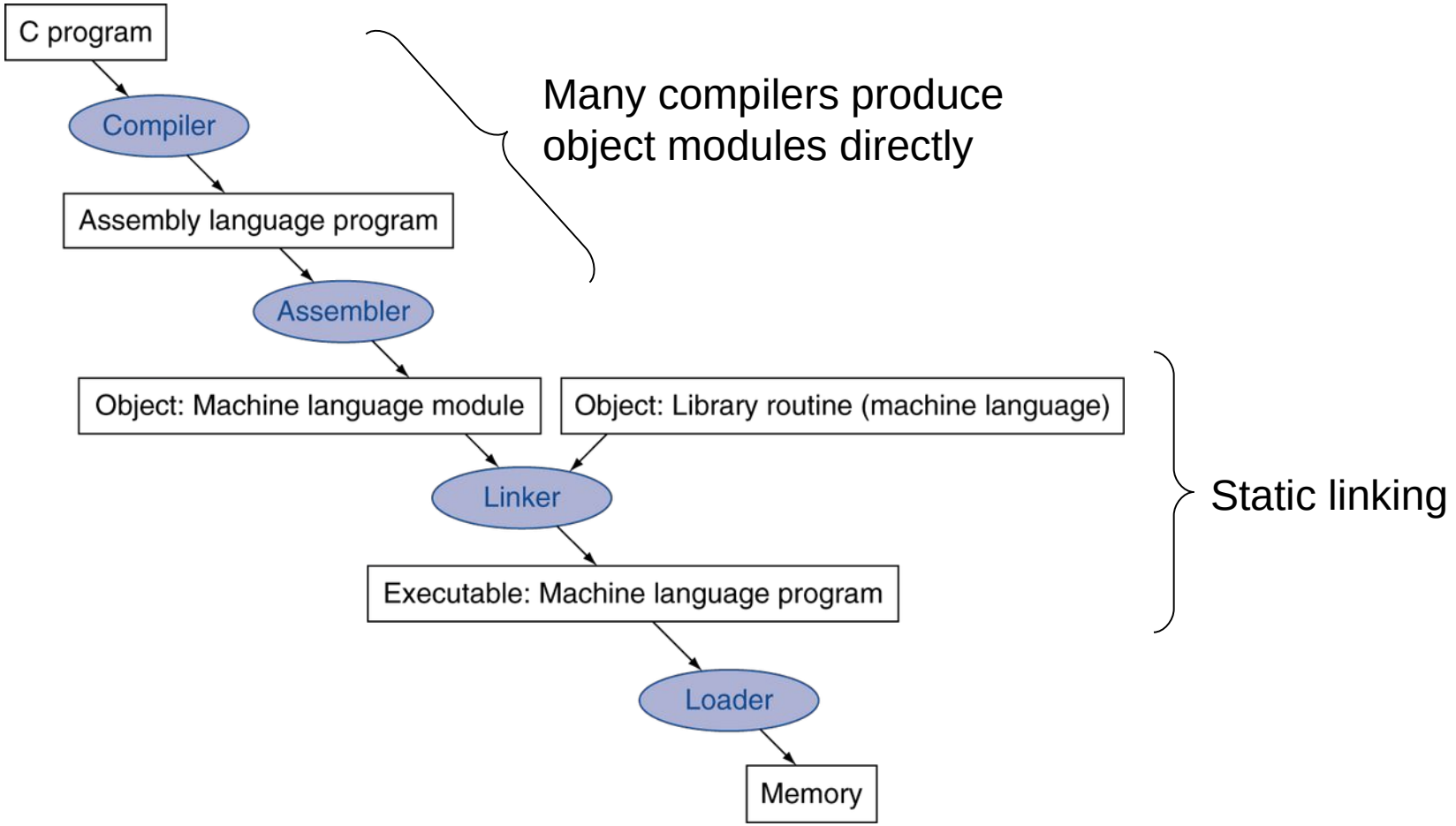
# Chapter 2

Instructions: Language of the Computer

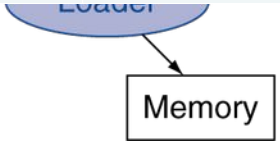
# Translation and Startup



# Translation and Startup



| Stage | Input | Translator / Tool | Output |
|---|---|---|---|
| **High-level Language → Assembly** | C / C++ code | **Compiler** | Assembly instructions |
| **Assembly → Machine Code** | MIPS assembly | **Assembler** | Machine instructions / Object module |
| **Object Modules → Executable** | Object modules + libraries | **Linker** | Executable program |
| **Executable → Memory** | Executable program | **Loader** | Program loaded into memory |



# Assembler Pseudoinstructions

- Most assembler instructions represent machine instructions one-to-one
- Pseudoinstructions: figments of the assembler's imagination

```
move $t0, $t1          → add $t0, $zero, $t1
blt $t0, $t1, L        → slt $at, $t0, $t1
                         bne $at, $zero, L
```

  - $at (register 1): assembler temporary

# Assembler Pseudoinstructions

- Most assembler instructions represent machine instructions one-to-one

**Q1. Translate each MIPS pseudoinstruction into actual MIPS instruction.**

**(a)neg $t0, $t1**

**(b)not $t0, $t1**

**(c)bgt $t0, $t1, L**

**A1.**

**(a)sub $t0, $zero, $t1**

**(b) nor $t0, $t1, $zero**

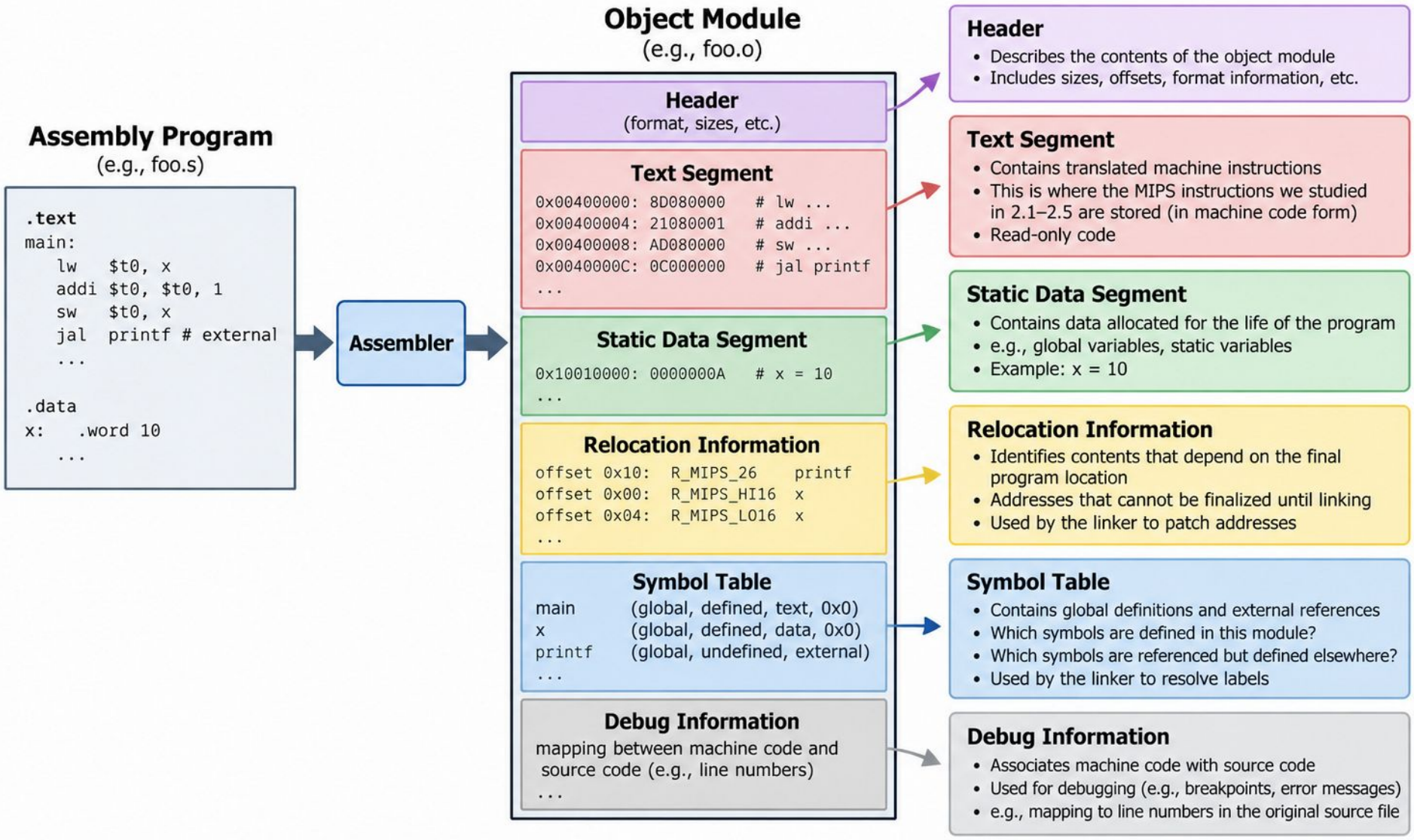
**(c) slt $at, $t1, $t0**

**bne $at, $zero, L**

# Producing an Object Module

- Assembler (or compiler) translates program into machine instructions
- Provides information for building a complete program from the pieces
  - Header: described contents of object module
  - Text segment: translated instructions
  - Static data segment: data allocated for the life of the program
  - Relocation info: for contents that depend on absolute location of loaded program
  - Symbol table: global definitions and external refs
  - Debug info: for associating with source code

# Producing an Object Module



# Linking Object Modules

- Produces an executable image
  1. Merges segments
  2. Resolve labels (determine their addresses)
  3. Patch location-dependent and external refs
- Could leave location dependencies for fixing by a relocating loader
  - But with virtual memory, no need to do this
  - Program can be loaded into absolute location in virtual memory space

# Linking Object Modules (Example)

## 1 Merge segments

- Combine text segments from all object modules
- Combine data segments from all object modules

**Before linking: separate object modules**

**main.o**

**Text segment**

```
0x0000:  li   $a0, 5
0x0004:  li   $a1, 3
0x0008:  jal  addTwo
0x000C:  move $s0, $v0
...
```

**Data segment**

```
...
```

**Symbol table**

```
main  (defined)
addTwo  (undefined)
```

**add.o**

**Text segment**

```
0x0000:  add  $v0,$a0,$a1
0x0004:  jr   $ra
...
```

**Data segment**

```
...
```

**Symbol table**

```
addTwo  (defined)
```

**After Step 1: merged segments (layout)**

| | | |
|---|---|---|
| 0x00400000 | main.o (text) | add.o (text) |
| 0x10010000 | main.o (data) | add.o (data) |

## 2 Resolve labels

- Use symbol tables to find the address of each global symbol
- Determine final addresses based on the merged layout

**Symbols after Step 2: final addresses**

**Text segment** (start at 0x00400000)

```
0x00400000    main      (from main.o)
0x00400020    addTwo    (from add.o)
              ...
```

**Data segment** (start at 0x10010000)

```
0x10010000    (main.o data)
0x10010020    (add.o data)
              ...
```

**Symbol table (relevant entries)**

| Symbol | Final Address |
|---|---|
| main | 0x00400000 |
| addTwo | 0x00400020 |
| ... | ... |

## 3 Patch references

- Update instructions/data that depend on symbol addresses
- Replace external references with the resolved addresses

**Example: patching the jal instruction**

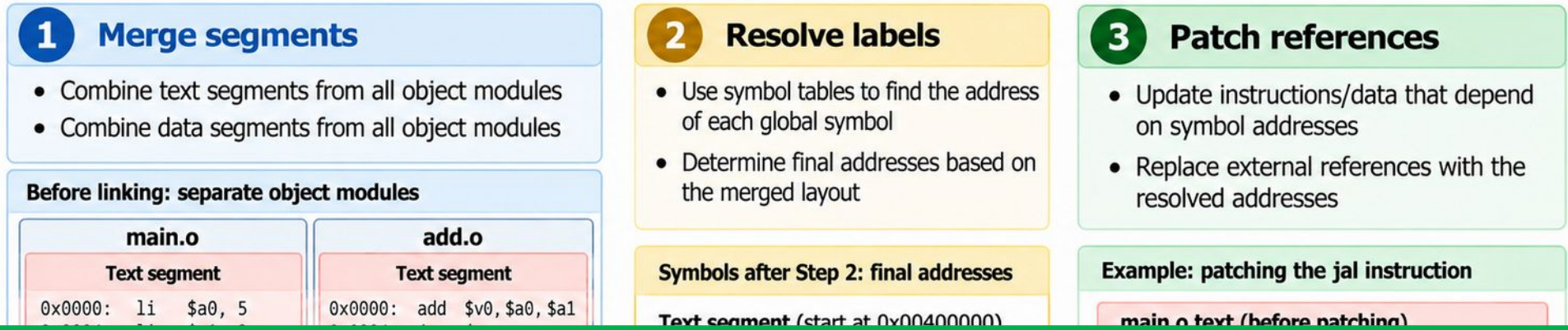
**main.o text (before patching)**

```
0x00400000:   li   $a0, 5
0x00400004:   li   $a1, 3
0x00400008:   jal  addTwo    # unresolved
0x0040000C:   move $s0, $v0
 ...
```

**main.o text (after patching)**

```
0x00400000:   li   $a0, 5
0x00400004:   li   $a1, 3
0x00400008:   jal  0x00400020  # patched
0x0040000C:   move $s0, $v0
 ...
```

# Linking Object Modules (Example)




**Q2. Suppose main.o contains a reference to addTwo, which is defined in another object module add.o. Which linking step determines the final address of addTwo, and which step updates the reference in main.o?**

**A2.**

**Resolve labels → determines the final address of addTwo.**

**Patch references → updates the reference in main.o using the resolved address.**

# C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(int v[], int k)
{
  int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

  - v in $a0, k in $a1, temp in $t0

# The Procedure Swap

```
swap: sll $t1, $a1, 2   # $t1 = k * 4
      add $t1, $a0, $t1 # $t1 = v+(k*4)
                        #   (address of v[k])
      lw $t0, 0($t1)    # $t0 (temp) = v[k]
      lw $t2, 4($t1)    # $t2 = v[k+1]
      sw $t2, 0($t1)    # v[k] = $t2 (v[k+1])
      sw $t0, 4($t1)    # v[k+1] = $t0 (temp)
      jr $ra            # return to calling routine
```

# The Procedure Swap

**Q3. Given the following C statement:**

```
x = A[i] + A[i+1];
```

**Assume: A → $s0 (base address of array A)**

**i → $s1, x → $s2**

**Translate the C statement into MIPS assembly**

**A3.**

```
sll  $t0, $s1, 2      # $t0 = i × 4
add  $t0, $s0, $t0    # $t0 = address of A[i]
lw   $t1, 0($t0)      # $t1 = A[i]
lw   $t2, 4($t0)      # $t2 = A[i+1]
add  $s2, $t1, $t2    # x = A[i] + A[i+1]
```

# ARM & MIPS Similarities

- ARM: the most popular embedded core
- Similar basic set of instructions to MIPS

| | ARM | MIPS |
|---|---|---|
| Date announced | 1985 | 1985 |
| Instruction size | 32 bits | 32 bits |
| Address space | 32-bit flat | 32-bit flat |
| Data alignment | Aligned | Aligned |
| Data addressing modes | 9 | 3 |
| Registers | 15 × 32-bit | 31 × 32-bit |
| Input/output | Memory mapped | Memory mapped |

# ARM & MIPS Similarities



- ARM: the most popular embedded core

| | MIPS | ARM |
|---|---|---|
| Registers | Simple organization | More flexible usage |
| Impact | Easy to manage | More operand flexibility |
| Data Addressing Modes | Simple addressing | More flexible addressing |
| Impact | May need more instructio ns | May reduce instructions |
| Advantages | Simple and regular | Flexible and widely supp orted |
| Limitations | Less flexible; smaller eco system | More complex ISA featur es |
| Typical Uses | Education, legacy embed ded | Mobile, embedded, PC, s ervers |
| Input/output | Memory mapped | Memory mapped |

# ARM & MIPS Similarities

- ARM: the most popular embedded core
- Similar basic set of instructions to MIPS

| | ARM | MIPS |
|---|---|---|

**Q4. A team wants to build a small embedded device that performs the same basic data-processing task. Which architecture would you prefer in each case? Briefly explain why.**

**(a)Used for teaching.**

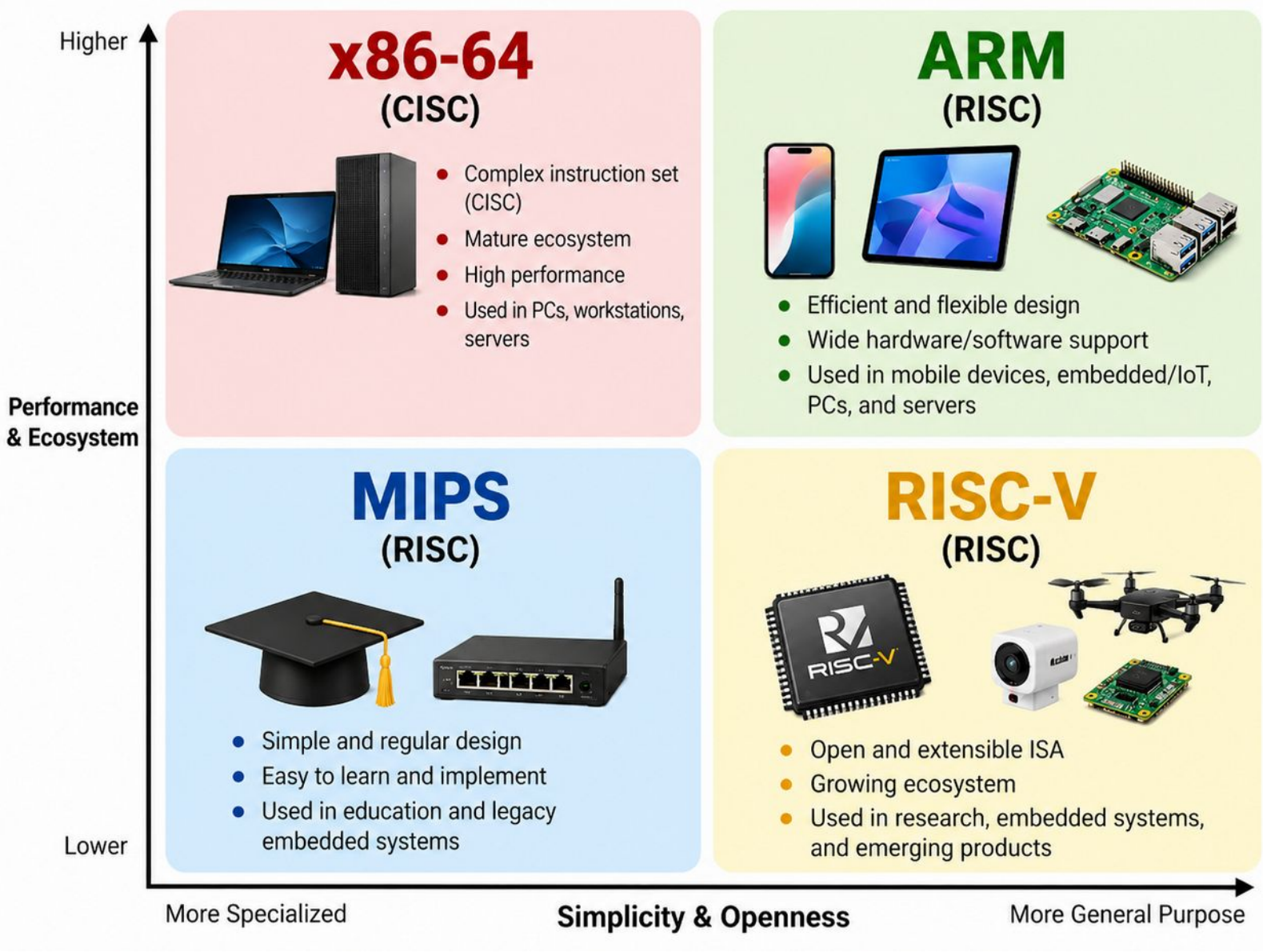
**(b) Designed as a modern commercial product**

**A4.**

**(a)MIPS — Simple and regular ISA.**

**(b) ARM — Flexible design and broad ecosystem.**

# MIPS vs. Other ISAs



# Fallacies

- Powerful instruction ⇒ higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code ⇒ more errors and less productivity

# Fallacies

- Powerful instruction ⇒ higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement

**Example:** Compute C = A + B[ i ] and store to memory

**Complex ISA (e.g., x86)**
1 complex instruction

```
ADD  [R1 + R2], R3
```

1 instruction

**Simple ISA (e.g., MIPS)**
3 simple instructions

```
lw   R4, 0(R2)
add  R4, R4, R3
sw   R4, 0(R1)
```

3 instructions

**Fewer instructions ≠ shorter execution time.**

Performance depends on
**Instruction count × CPI × Clock cycle time**

**Example:** Compute the sum of an array

**High-level language (C)**
Simple and concise

```
int sum = 0;
for (int i = 0; i < N; i++)
    sum += A[i];
```

**Assembly code (e.g., MIPS)**
Many low-level instructions

```
  li   $t0, 0          # sum = 0
  li   $t1, 0          # i = 0
loop:
  lw   $t2, 0($t3)     # load A[i]
  add  $t0, $t0, $t2   # sum += A[i]
  addi $t3, $t3, 4     # next address
  addi $t1, $t1, 1     # i++
  blt  $t1, $t4, loop  # if i < N
```

- Modern compilers can generate **highly optimized code.**
- Manual assembly is **harder to write, debug, and maintain.**
- It is also **less portable** across different **architectures.**

# Concluding Remarks



- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands good compromises
- Layers of software/hardware
  - Compiler, assembler, hardware
- MIPS: typical of RISC ISAs
  - c.f. x86

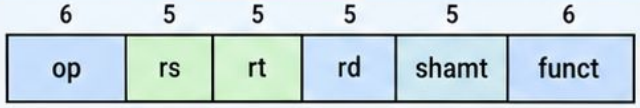
# Concluding Remarks

- Measure MIPS instruction executions in benchmark programs
  - Consider making the common case fast
  - Consider compromises

| Instruction class | MIPS examples | SPEC2006 Int | SPEC2006 FP |
|---|---|---|---|
| Arithmetic | `add, sub, addi` | 16% | 48% |
| Data transfer | `lw, sw, lb, lbu, lh, lhu, sb, lui` | 35% | 36% |
| Logical | `and, or, nor, andi, ori, sll, srl` | 12% | 4% |
| Cond. Branch | `beq, bne, slt, slti, sltiu` | 34% | 8% |
| Jump | `j, jr, jal` | 2% | 0% |

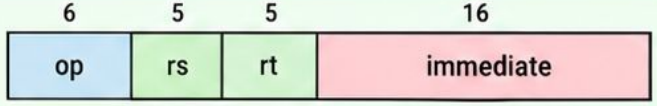
# MIPS Instructions Learned in Chapter 2

## R-format (Register)




| Category | Instruction | Description |
|---|---|---|
| Arithmetic | `add, sub` | Addition, subtraction |
| Logical | `and, or, nor` | Bitwise AND, OR, NOR |
| Shift | `sll, srl` | Logical shift left/right |
| Comparison | `slt, sltu` | Set on less than (signed/unsigned) |
| Jump | `jr` | Jump to register |

## I-format (Immediate / Memory / Branch)




| Category | Instruction | Description |
|---|---|---|
| Arithmetic | `addi` | Add immediate (sign-extended) |
| Logical | `andi, ori` | Bitwise AND/OR with immediate (zero-extended) |
| Data Transfer | `lw, sw` | Load word, store word |
| | `lb, lbu` | Load byte (signed/unsigned) |
| | `lh, lhu` | Load half word (signed/unsigned) |
| | `sb` | Store byte |
| | `lui` | Load upper immediate |
| Branch | `beq, bne` | Branch if equal / not equal |
| Comparison | `slti, sltiu` | Set on less than immediate (signed/unsigned) |

## J-format (Jump)




| Instruction | Description |
|---|---|
| `j` | Jump to target address |
| `jal` | Jump and link (store return address in $ra) |

## Pseudoinstructions (Assembler Convenience)

| Pseudoinstruction | Meaning | Translated Using (Real Instructions) | Example |
|---|---|---|---|
| `move rd, rs` | Copy register | `add rd, rs, $zero` | `move $t0, $t1 → add $t0, $t1, $zero` |
| `blt rs, rt, label` | Branch if less than (signed) | `slt $at, rs, rt`<br>`bne $at, $zero, label` | `blt $t0, $t1, L → slt $at, $t0, $t1`<br>`bne $at, $zero, L` |
| `bgt rs, rt, label` | Branch if greater than (signed) | `slt $at, rt, rs`<br>`bne $at, $zero, label` | `bgt $t0, $t1, L → slt $at, $t1, $t0`<br>`bne $at, $zero, L` |
| `neg rd, rs` | Negation | `sub rd, $zero, rs` | `neg $t0, $t1 → sub $t0, $zero, $t1` |
| `not rd, rs` | Bitwise NOT | `nor rd, rs, $zero` | `not $t0, $t1 → nor $t0, $t1, $zero` |

# Q5. Complete the following table. For each MIPS operation, write its full name/meaning, instruction format, and one valid example instruction.

| Operation | Full Name / Meaning | Format | Example Instruction |
|---|---|---|---|
| add | | | |
| lw | | | |
| slt | | | |
| beq | | | |
| jal | | | |
| lbu | | | |
| lui | | | |
| sltu | | | |
| ori | | | |
| jr | | | |

# Q5. Complete the following table. For each MIPS operation, write its full name/meaning, instruction format, and one valid example instruction.

| Operation | Full Name / Meaning | Format | Example Instruction |
|---|---|---|---|
| add | Add | | |
| lw | Load Word | | |
| slt | Set on Less Than | | |
| beq | Branch if Equal | | |
| jal | Jump and Link | | |
| lbu | Load Byte Unsigned | | |
| lui | Load Upper Immediate | | |
| sltu | Set on Less Than Unsigned | | |
| ori | OR Immediate | | |
| jr | Jump Register | | |

# Q5. Complete the following table. For each MIPS operation, write its full name/meaning, instruction format, and one valid example instruction.

| Operation | Full Name / Meaning | Format | Example Instruction |
|---|---|---|---|
| add | Add | R-format | |
| lw | Load Word | I-format | |
| slt | Set on Less Than | R-format | |
| beq | Branch if Equal | I-format | |
| jal | Jump and Link | J-format | |
| lbu | Load Byte Unsigned | I-format | |
| lui | Load Upper Immediate | I-format | |
| sltu | Set on Less Than Unsigned | R-format | |
| ori | OR Immediate | I-format | |
| jr | Jump Register | R-format | |

# Q5. Complete the following table. For each MIPS operation, write its full name/meaning, instruction format, and one valid example instruction.

| Operation | Full Name / Meaning | Format | Example Instruction |
|---|---|---|---|
| add | Add | R-format | add $t0, $t1, $t2 |
| lw | Load Word | I-format | lw $t0, 4($s0) |
| slt | Set on Less Than | R-format | slt $t0, $t1, $t2 |
| beq | Branch if Equal | I-format | beq $t0, $t1, L1 |
| jal | Jump and Link | J-format | jal procedure |
| lbu | Load Byte Unsigned | I-format | lbu $t0, 0($s0) |
| lui | Load Upper Immediate | I-format | lui $t0, 100 |
| sltu | Set on Less Than Unsigned | R-format | sltu $t0, $t1, $t2 |
| ori | OR Immediate | I-format | ori $t0, $t1, 15 |
| jr | Jump Register | R-format | jr $ra |

# Section 2-6 Quick Questions

**Q1.** Translate neg, not, and bgt pseudoinstructions into actual MIPS instructions.

**Q2.** For addTwo defined in another object module, which linking steps resolve its address and update the reference?

**Q3.** Translate x = A[i] + A[i+1] into MIPS assembly.

**Q4.** Choose MIPS or ARM for teaching and for a modern commercial product. Briefly explain why.

**Q5.** For 10 given MIPS operations, identify the full name/meaning, format, and one valid example instruction.

# Exercise 1 | Pseudoinstruction Expansion

A programmer writes:

```
move $t0, $s0
bgt  $t0, $s1, L1
```

Assume $at can be used by the assembler.

(a) Expand both pseudoinstructions into actual MIPS instructions.

(b) How many actual MIPS instructions are generated in total?

(c) Which register is used by the assembler when expanding bgt?

## Answer

(a) move $t0, $s0 → add $t0, $zero, $s0

bgt $t0, $s1, L1 →

```
slt $at, $s1, $t0
bne $at, $zero, L1
```

(b) 3 actual MIPS instructions

(c) $at — assembler temporary

**Key Point:** One pseudoinstruction may expand into one or more actual MIPS instructions.

# Exercise 2 | Procedure Swap Analysis

```
swap:
    sll  $t1, $a1, 2
    add  $t1, $a0, $t1
    lw   $t0, 0($t1)
    lw   $t2, 4($t1)
    sw   $t2, 0($t1)
    sw   $t0, 4($t1)
    jr   $ra
```

Assume $a0 = 1000, $a1 = 2, Memory[1008] = 15, and Memory[1012] = 25.

(a) What address is calculated in $t1?

(b) What values are loaded into $t0 and $t2?

(c) What are Memory[1008] and Memory[1012] after the procedure finishes?

## Answer

(a) $t1 = 2 × 4 + 1000 = 1008

(b) $t0 = 15, $t2 = 25

(c) Memory[1008] = 25, Memory[1012] = 15

**Key Point:** The swap procedure combines array addressing, load/store instructions, and procedure return in one complete example.

# Exercise 3 | Trace a MIPS Loop

**Assume:**
**$s0 = 1000, $s1 = 0, $s2 = 3**
**Memory[1000] = 5, Memory[1004] = 8, Memory[1008] = 12**

```
loop:  lw   $t0, 0($s0)
       add  $s1, $s1, $t0
       addi $s0, $s0, 4
       addi $s2, $s2, -1
       bne  $s2, $zero, loop
```

(a) How many iterations are executed?
(b) What values are loaded into $t0?
(c) What are the final values of $s0, $s1, and $s2?

## Answer

(a) 3 iterations
(b) $t0: 5 → 8 → 12
(c) $s0 = 1012, $s1 = 25, $s2 = 0

**Key Point:** Tracing a loop requires following memory accesses, register updates, and branch conditions across iterations.

# Exercise 4 | Integrated Instruction Analysis

**Given:**

```
lw   $t0, 8($s0)
add  $t1, $t0, $s1
slt  $t2, $t1, $s2
bne  $t2, $zero, L1
jal  update
```

(a) Classify each instruction as Data Transfer, Arithmetic, Comparison/Branch, or Jump/Procedure.
(b) Identify the format of each instruction: R, I, or J.
(c) Which instruction accesses memory, performs the comparison, and calls a procedure?

## Answer

lw → Data Transfer, I-format
add → Arithmetic, R-format
slt → Comparison, R-format
bne → Conditional Branch, I-format
jal → Jump / Procedure, J-format
(c) Memory access → lw, Comparison → slt, Procedure call → jal

**Key Point:** A MIPS program combines different instruction classes and formats for data, computation, control flow, and procedures.